
Guide Essentiel des Bases de Données SQL

Modèle relationnel, requêtes SELECT, jointures, contraintes, transactions, index et bonnes pratiques

Rapport pédagogique — 8 pages

1. Introduction aux bases de données relationnelles

Une base de données relationnelle organise les informations en **tables** (ou relations), composées de **lignes** (enregistrements) et de **colonnes** (attributs). Ce modèle, proposé par Edgar Codd en 1970, reste la référence pour la majorité des systèmes de gestion de bases de données (SGBD).

SGBD relationnels courants

MySQL / MariaDB	Très répandu pour le web, open source, performant en lecture.
PostgreSQL	Open source, très complet, respecte fortement le standard SQL.
SQLite	Base de données légère, fichier unique, embarquée dans les applications.
SQL Server / Oracle	Solutions propriétaires orientées entreprise, riches en fonctionnalités.

Concepts fondamentaux

- **Table** : ensemble structuré de données sur un même sujet (ex : Clients, Commandes).
- **Clé primaire (PK)** : identifiant unique de chaque ligne d'une table.
- **Clé étrangère (FK)** : référence vers la clé primaire d'une autre table, créant une relation.
- **Schéma** : structure complète d'une base (tables, colonnes, types, contraintes).

Le langage SQL

SQL (Structured Query Language) se divise en plusieurs sous-langages : **DDL** (Data Definition Language : CREATE, ALTER, DROP), **DML** (Data Manipulation Language : INSERT, UPDATE, DELETE), **DQL** (Data Query Language : SELECT) et **DCL** (Data Control Language : GRANT, REVOKE).

2. Créer et structurer une table

L'instruction `CREATE TABLE` définit la structure d'une table : noms de colonnes, types de données et contraintes associées.

```
CREATE TABLE clients (  
    id            INT PRIMARY KEY AUTO_INCREMENT,  
    nom           VARCHAR(100) NOT NULL,  
    email         VARCHAR(150) UNIQUE NOT NULL,  
    date_inscription DATE DEFAULT CURRENT_DATE  
);  
  
CREATE TABLE commandes (  
    id            INT PRIMARY KEY AUTO_INCREMENT,  
    client_id     INT NOT NULL,  
    montant       DECIMAL(10, 2) NOT NULL,  
    statut        VARCHAR(20) DEFAULT 'en attente',  
    FOREIGN KEY (client_id) REFERENCES clients(id)  
);
```

Types de données courants

INT / BIGINT	Nombres entiers, avec ou sans limite étendue.
DECIMAL(p, d)	Nombre décimal exact — indispensable pour les montants.
VARCHAR(n)	Chaîne de caractères de longueur variable, jusqu'à n.
DATE / DATETIME	Date seule, ou date et heure.
BOOLEAN	Valeur vraie/fausse (souvent stockée en TINYINT(1)).

Modifier et insérer des données

```
ALTER TABLE clients ADD COLUMN telephone VARCHAR(20);  
  
INSERT INTO clients (nom, email) VALUES ('Amine El Idrissi', 'amine@mail.com');  
  
UPDATE clients SET telephone = '0600000000' WHERE id = 1;  
  
DELETE FROM clients WHERE id = 5;
```

3. Requêtes SELECT et filtrage

L'instruction SELECT est au cœur du SQL : elle permet d'extraire, filtrer, trier et agréger des données.

```
-- Sélection simple avec filtre et tri

SELECT nom, email

FROM clients

WHERE date_inscription >= '2025-01-01'

ORDER BY nom ASC;


-- Filtres combinés

SELECT * FROM commandes

WHERE statut = 'en attente' AND montant > 100;


-- Motif de recherche et intervalle

SELECT nom FROM clients WHERE nom LIKE 'A%';

SELECT * FROM commandes WHERE montant BETWEEN 50 AND 200;
```

Agrégations et regroupement

Les fonctions d'agrégat (COUNT, SUM, AVG, MIN, MAX) résument un ensemble de lignes, souvent combinées à GROUP BY pour un calcul par groupe.

```
SELECT statut, COUNT(*) AS nombre, SUM(montant) AS total

FROM commandes

GROUP BY statut

HAVING COUNT(*) > 1

ORDER BY total DESC;
```

HAVING filtre les groupes après agrégation, alors que WHERE filtre les lignes avant regroupement.

4. Les jointures (JOIN)

Une jointure combine les lignes de plusieurs tables selon une condition de correspondance, typiquement entre une clé primaire et une clé étrangère.

INNER JOIN	Ne conserve que les lignes ayant une correspondance dans les deux tables.
LEFT JOIN	Conserve toutes les lignes de la table de gauche, avec NULL si pas de correspondance.
RIGHT JOIN	Conserve toutes les lignes de la table de droite, symétrique du LEFT JOIN.
FULL OUTER JOIN	Conserve toutes les lignes des deux tables, correspondance ou non.

```
-- Clients et leurs commandes (uniquement s'ils en ont)

SELECT c.nom, o.id AS commande_id, o.montant
FROM clients c
INNER JOIN commandes o ON c.id = o.client_id;

-- Tous les clients, même sans commande

SELECT c.nom, o.montant
FROM clients c
LEFT JOIN commandes o ON c.id = o.client_id;
```

Jointure multiple

```
SELECT c.nom, p.nom AS produit, l.quantite
FROM clients c
JOIN commandes o ON c.id = o.client_id
JOIN lignes_commande l ON o.id = l.commande_id
JOIN produits p ON l.produit_id = p.id;
```

5. Contraintes et normalisation

Contraintes d'intégrité

PRIMARY KEY	Identifie de façon unique chaque ligne ; ne peut pas être NULL.
FOREIGN KEY	Garantit qu'une valeur existe bien dans la table référencée.
UNIQUE	Interdit les doublons sur une colonne (sans être forcément la clé primaire).
NOT NULL	Interdit qu'une colonne reste vide.
CHECK	Impose une condition sur les valeurs (ex : CHECK (montant > 0)).

Normalisation : pourquoi diviser les tables ?

La normalisation limite la redondance et les anomalies de mise à jour, en organisant les données selon des règles progressives appelées formes normales (FN).

- **1FN** : chaque colonne contient une valeur atomique (pas de listes dans une cellule).
- **2FN** : respecte la 1FN et chaque colonne non-clé dépend de la totalité de la clé primaire.
- **3FN** : respecte la 2FN et aucune colonne non-clé ne dépend d'une autre colonne non-clé.

```
-- Non normalisé : redondance du nom du produit à chaque ligne
commandes(id, client_nom, produit_nom, produit_prix, quantite)

-- Normalisé : séparation en entités distinctes reliées par des clés
clients(id, nom)
produits(id, nom, prix)
lignes_commande(commande_id, produit_id, quantite)
```

6. Transactions et index

Transactions et propriétés ACID

Une transaction regroupe plusieurs instructions en une seule unité atomique : soit toutes sont appliquées, soit aucune ne l'est. Les SGBD garantissent les propriétés **ACID** : Atomicité, Cohérence, Isolation, Durabilité.

```
START TRANSACTION;

UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;

-- Si tout est correct :
COMMIT;

-- En cas d'erreur détectée :
ROLLBACK;
```

Index : accélérer les recherches

Un index est une structure (souvent un arbre B) qui accélère la recherche sur une colonne, au prix d'un espace disque et d'un coût supplémentaire à l'écriture.

```
CREATE INDEX idx_clients_email ON clients(email);

-- Vérifier si une requête utilise un index :
EXPLAIN SELECT * FROM clients WHERE email = 'amine@mail.com';
```

Indexer une colonne peu sélective (ex : un booléen) ou trop de colonnes peut ralentir les écritures sans réel bénéfice en lecture.

7. Sous-requêtes, vues et bonnes pratiques

Sous-requêtes (subqueries)

```
-- Clients ayant passé au moins une commande de plus de 200

SELECT nom FROM clients

WHERE id IN (

    SELECT client_id FROM commandes WHERE montant > 200

);

-- Comparaison avec une valeur agrégée

SELECT nom, montant FROM commandes

WHERE montant > (SELECT AVG(montant) FROM commandes);
```

Vues (VIEW)

Une vue est une requête SQL enregistrée, utilisable comme une table virtuelle : elle simplifie les requêtes complexes et peut restreindre l'accès à certaines colonnes.

```
CREATE VIEW commandes_recentes AS

SELECT c.nom, o.montant, o.statut

FROM clients c

JOIN commandes o ON c.id = o.client_id

WHERE o.statut != 'annulee';

SELECT * FROM commandes_recentes WHERE montant > 100;
```

Bonnes pratiques

- Toujours définir une clé primaire et des contraintes de clé étrangère explicites.
- Éviter `SELECT *` en production : préférer lister les colonnes utiles.
- Utiliser des requêtes préparées pour éviter les injections SQL.
- Indexer les colonnes fréquemment utilisées dans `WHERE`, `JOIN` et `ORDER BY`.

Fin du guide — bonne pratique des bases de données SQL !